

CSE 423 — Compilers

Sample Final Exam

Instructor: Huixin Zhan

For practice before May 15

Note from your instructor. This sample exam is the same *shape* as the real final but the questions are slightly different. Each question here is followed immediately by a green “walkthrough” box. **I will explain the walkthrough live in class.** Use it as a model for the level of detail your real exam answers should reach.

Question 1. Symbol Tables (10 pts)

A symbol table is implemented as a singly linked list of entries. Each entry stores the symbol’s name and type. The two operations are:

```
struct symtab_entry { char *sym; struct typeinfo *type; };
struct elem { struct symtab_entry *ste; struct elem *next; };

struct symtab_entry *lookup(struct elem *st, char *name) {
    if (st == NULL) return NULL;
    if (!strcmp(st->ste->sym, name)) return st->ste;
    return lookup(st->next, name);
}

struct elem *insert(struct elem *st, char *name, struct typeinfo *t) {
    if (lookup(st, name) != NULL) {
        fprintf(stderr, "symbol is already inserted\n"); exit(3);
    }
    struct symtab_entry *ste = malloc(sizeof *ste);
    ste->sym = strdup(name); ste->type = t;
    struct elem *n = malloc(sizeof *n);
    n->ste = ste; n->next = st; /* prepends */
    return n;
}
```

Now consider the following declarations processed in order at the global scope, with `st` starting as `NULL`:

```
int x;
char c;
float f;
int *p;
```

(a) Describe *exactly* what happens on each of the four lines (which branches execute, what is allocated, what gets prepended). (b) Draw the final state of the linked list (head $\rightarrow$ tail) after

all four lines are processed. (c) In one sentence, explain why `insert` must call `lookup` *before* it allocates a new entry, rather than after.

In-class walkthrough (we will work this out together)

Walkthrough.

1. `int x;` – `lookup(NULL, "x")` immediately returns `NULL`. `insert` allocates a new `syntab_entry` with `sym="x"`, `type=int`, wraps it in an `elem`, and prepends. List: $[x : \text{int}]$.
2. `char c;` – `lookup` walks the one-node list, no match, returns `NULL`. Insert succeeds. List: $[c : \text{char}] \rightarrow [x : \text{int}]$.
3. `float f;` – `lookup` walks two nodes, no match, returns `NULL`. Insert succeeds. List: $[f : \text{float}] \rightarrow [c : \text{char}] \rightarrow [x : \text{int}]$.
4. `int *p;` – `lookup` returns `NULL`. The type “pointer to int” is constructed first (a `POINTER` type whose element type is `int`), and then `insert` prepends. Final list: $[p : \text{ptr} \rightarrow \text{int}] \rightarrow [f : \text{float}] \rightarrow [c : \text{char}] \rightarrow [x : \text{int}]$.

(c) If `insert` allocated first and only checked afterward, a duplicate name could be prepended at the head of the list before the check fired. Future calls to `lookup` (which scan head-to-tail) would return the *new* entry, masking the original; if the redeclaration uses a different type, every later type-check on that name silently uses the wrong type.

Question 2. Synthesized vs. Inherited Attributes (10 pts)

For the declaration

```
var p, q float
```

draw the parse tree and show, step by step, how the type `float` reaches each leaf. Label each step as *synthesized* or *inherited*.

In-class walkthrough (we will work this out together)

Walkthrough. Parse tree:

```
VARDCL
|-- DNL
|   |-- LNAME("p")
|   +-- LNAME("q")
+-- NTYPE
    +-- LNAME("float")
```

Step 1 (synthesized). LNAME("float") produces T_FLOAT; NTYPE copies it up. Now NTYPE.type = float.

Step 2 (synthesized). VARDCL.type = kid[1].type = float.

Step 3 (inherited). VARDCL writes its type *down* into the left subtree: DNL.type = float.

Step 4 (inherited, recursive). DNL pushes float into both LNAME("p") and LNAME("q").

Step 5. At each LNAME leaf, call `st_insert(st, name, type)` producing the symbol table entries `p:float` and `q:float`.

Why two directions? The type lives in one subtree, but the names live in another. Synthesized attributes lift the type up to the common ancestor; inherited attributes push it down to the leaves that need it.

Question 3. Three-Address Code Translation (15 pts)

Translate the following C-like fragment into Three-Address Code (TAC). Use temporaries $t_1, t_2, \dots$ as needed and follow the opcode set from Lecture 16.

```
a = (b + c) * (d - e);
if (a > 0) a = a + 1;
```

In-class walkthrough (we will work this out together)

Walkthrough. Each binary operator becomes one TAC instruction. Allocate one temporary per intermediate value:

```
t1 = b + c
t2 = d - e
t3 = t1 * t2
a = t3
t4 = a > 0
BIFN t4, Lend ; if !t4 goto Lend
t5 = a + 1
a = t5
lab Lend
```

Notes. (1) The condition `a > 0` produces a temporary `t4`; we then branch on its negation using BIFN. (2) We could also use `BLE a, 0, Lend` directly – both forms are accepted by the Lecture 16 opcode set. (3) The label `Lend` is what closes the “if” construct.

Question 4. Basic Block Partitioning (10 pts)

Partition the following TAC into basic blocks. For each block, list its instructions and identify its successors.

```
1: read n B1 starts here (rule 1: first instruction)
2: i = 1
3: s = 0
4: lab L1 B2 starts here (rule 2: label)
5: t1 = i > n
6: BIF t1, L2
7: s = s + i B3 starts here (rule 3: right after BIF)
8: i = i + 1
9: goto L1
10: lab L2 B4 starts here (rule 2: label, AND rule 3: after goto)
11: print s
```

In-class walkthrough (we will work this out together)

Walkthrough. Apply the boundary rules: a block starts at line 1, at every label, and immediately after every jump.

- **Block B1** (lines 1–3): `read n`; `i=1`; `s=0`. Successor: B2 (fall-through into the label).
- **Block B2** (lines 4–6): `lab L1`; `t1=i>n`; `BIF t1,L2`. Successors: B4 (when `t1` is true, jumps to L2) and B3 (fall-through).
- **Block B3** (lines 7–9): `s=s+i`; `i=i+1`; `goto L1`. Successor: B2 (back-edge – this is the loop).
- **Block B4** (lines 10–11): `lab L2`; `print s`. Successor: exit.

How to determine the successors For each block, look at its last instruction. That tells you where control can go next:

Last instruction is an unconditional `goto L` → successor is the block whose label is L. Just one successor.

Last instruction is a conditional branch like `BIF t1, L2` → two successors: the block at label L2 (if the condition is true and we jump), AND the next block in sequence (if the condition is false and we just fall through).

Last instruction is plain (no jump, no branch) → successor is just the next block in sequence (fall-through).

Last instruction is return or end of program → no successor (or "exit").

Applying that to the example:

B1 ends with `s = 0` (plain). → successor: B2 (fall-through).

B2 ends with `BIF t1, L2` (conditional branch). → two successors: B4 (where L2 is) and B3 (fall-through).

B3 ends with `goto L1` (unconditional). → successor: B2 (where L1 is). This is the loop back-edge.

B4 ends with `print s` (last line). → successor: exit.

CFG sketch. B1 → B2 → B3 → B2 (loop), B2 → B4 → exit.

Question 5. DAG Construction and Common Subexpressions (15 pts)

Build a DAG for the following basic block. Then list every common subexpression it exposes.

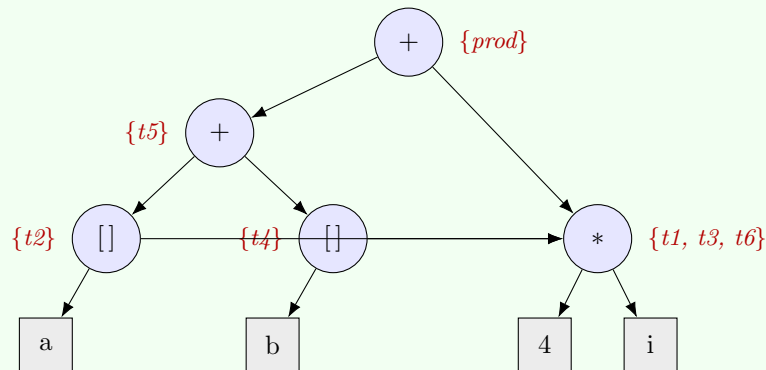
```
t1 = 4 * i
t2 = a[t1]
t3 = 4 * i
t4 = b[t3]
t5 = t2 + t4
t6 = 4 * i
prod = t5 + t6
```

In-class walkthrough (we will work this out together)

Walkthrough. Process each instruction in order. For each $x := y \text{ op } z$, look for an existing node with that operator and those operands; if found, reuse it.

- $t1 = 4 * i$: create leaves 4 and i ; create node $N_1 = *(4, i)$; $N_1.\text{ids} = \{t_1\}$.
- $t2 = a[t1]$: create leaf a ; create $N_2 = [](a, N_1)$; $N_2.\text{ids} = \{t_2\}$.
- $t3 = 4 * i$: a node $*(4, i)$ already exists – reuse N_1 . $N_1.\text{ids} = \{t_1, t_3\}$.
- $t4 = b[t3]$: create leaf b ; create $N_3 = [](b, N_1)$; $N_3.\text{ids} = \{t_4\}$.
- $t5 = t2 + t4$: create $N_4 = +(N_2, N_3)$; $N_4.\text{ids} = \{t_5\}$.
- $t6 = 4 * i$: reuse N_1 again. $N_1.\text{ids} = \{t_1, t_3, t_6\}$.
- $\text{prod} = t5 + t6$: create $N_5 = +(N_4, N_1)$; $N_5.\text{ids} = \{\text{prod}\}$.

The DAG:



Notice that the node $N_1 = 4*i$ has **three** incoming edges (from N_2 , N_3 , and N_5). That is exactly the visual signature of a common subexpression: one node, multiple references.

Common subexpression detected: $4*i$ is computed once at N_1 and reused by t_1 , t_3 , and t_6 . Optimized code only needs the multiplication once:

```
t1 = 4 * i
t2 = a[t1]
t4 = b[t1]
t5 = t2 + t4
prod = t5 + t1
```

We saved *two* multiplications per loop iteration. This is exactly the kind of redundancy that array indexing creates implicitly.

Question 6. Naming the Optimization (15 pts)

For each of the rewrites below, name the optimization technique being applied (one of: constant folding, common subexpression elimination, loop unrolling, strength reduction, hoisting loop invariants, peephole, interprocedural). One word answers are fine, but justify briefly.

- (a) $x = 3 + 4 * 5 \rightarrow x = 23$
- (b) $y = z * 8 \rightarrow y = z \ll 3$
- (c) `for(i=0;i<n;i++) a[i] = strlen(s) + i; → int L = strlen(s); for(...) a[i] = L + i;`
- (d) `movq %rax, -8(%rbp); cmpq $0, -8(%rbp) → cmpq $0, %rax`
- (e) An inlined call across translation units after whole-program analysis.

In-class walkthrough (we will work this out together)

Walkthrough.

- (a) **Constant folding.** Both operands are compile-time constants, so the compiler computes the result at compile time.
- (b) **Strength reduction** (an algebraic-identity optimization). Multiplying by a power of two becomes a left shift, which is faster.
- (c) **Hoisting loop invariants.** `strlen(s)` does not depend on `i`, so it is moved out of the loop and computed once.
- (d) **Peephole.** The window of two adjacent instructions sees that the just-stored value is still in `%rax`, so the reload is dead.
- (e) **Interprocedural optimization (IPO).** Inlining across function boundaries requires whole-program (or LTO) information.

Question 7. Reading x86-64 from gcc -S (10 pts)

Below is the assembly produced by `gcc -S -O0` for a small function. Reconstruct the C source. Explain the role of `%rdi`, `%rsi`, and `%rax`.

```
add:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl -4(%rbp), %edx
    movl -8(%rbp), %eax
    addl %edx, %eax
    ret
```

In-class walkthrough (we will work this out together)

Walkthrough.

- Two arguments arrive in `%edi` and `%esi` (the System V AMD64 calling convention puts the first two integer arguments in `%rdi/%rsi`; the lower 32 bits are `%edi/%esi`).
- They are spilled to the stack at `-4(%rbp)` and `-8(%rbp)`.
- Both values are loaded back into `%edx` and `%eax`, added, and the result remains in `%eax` – the integer return register.
- Reconstructed source:

```
int add(int a, int b) { return a + b; }
```

Why the back-and-forth? `-O0` disables optimizations to keep behavior debuggable. With `-O2` the entire body would collapse to `lea (%rdi,%rsi), %eax; ret.`

Question 8. Putting It All Together (15 pts)

Consider the C fragment

```
for (i = 0; i < 100; i++) {  
    a[i] = b[i] + 4 * i;  
}
```

- (a) Generate TAC for the loop body.
- (b) Identify two distinct optimization opportunities in your TAC and apply them.
- (c) Briefly explain why *loop unrolling* alone, without other optimizations following it, would not give the largest speedup.

In-class walkthrough (we will work this out together)

Walkthrough.

(a) TAC for the body.

```
t1 = 4 * i
t2 = &b
t3 = t2 + t1 ; address of b[i]
t4 = *t3 ; LCON: load b[i]
t5 = 4 * i ; recomputed index for a[i]
t6 = &a
t7 = t6 + t5
t8 = t4 + t1 ; b[i] + 4*i
*t7 = t8 ; SCON: store into a[i]
```

(b) Two optimizations.

- *Common subexpression elimination.* $4*i$ is computed twice ($t1$, $t5$); reuse $t1$.
- *Strength reduction on the loop induction variable.* Instead of recomputing $4*i$ every iteration, maintain a running pointer/offset that increments by 4 each step. This replaces a multiply with an add per iteration.

(c) **Why unrolling alone is not enough.** Unrolling reduces loop overhead (the comparison, increment, and back-jump) but does *not* by itself reduce the work *inside* the body. The biggest wins come when unrolling *exposes* new opportunities – constants become foldable, addresses become merge-able, and the scheduler has more independent instructions to overlap. Without those follow-on passes, the unrolled body just contains four copies of the same expensive multiplications, and the speedup is modest.

End of sample exam. Total: 100 points across 8 questions. The real final will have the same structure but will perturb the specific code, types, and tree shapes – the underlying skills are identical.